MSc in Mathematics and Foundations of Computer Science

LAMBDA CALCULUS AND TYPES

Hilary Term 2026

Submission deadline 12 noon, Wednesday 15th April 2026, via Inspera.

There is a total of 100 marks available for this paper, you should attempt all parts of the paper.

NB: You must not discuss this examination paper with anyone.

TURN OVER

If some part requires you to prove a statement or construct a term, but you are unable to do so, you may nevertheless assume the statement or the existence of this term when answering later parts of the paper.

Justify all your answers.

Question 1

In this question, "reduction" refers to β-reduction, "normal form" to β-normal form and "=" to equality in $\lambda\beta$. When asked about the size of a term or reduction, an asymptotic characterisation suffices, e.g., $\mathcal{O}(n)$ for some given n.

We define an encoding of the natural numbers (different from the one in the lectures) as follows. For any $n \in \mathbb{N}$, $\lceil n \rceil$ is the normal form of the term defined by:

$$\lceil 0 \rceil \equiv \lambda sz.z$$
$$\lceil n+1 \rceil \equiv \lambda sz.s\lceil n \rceil(\lceil n \rceil s\ z)$$

(a) What is the size of $\lceil n \rceil$? (2 marks)

We call r a *recursor* if we have closed terms **zero** and **succ** such that for all terms s, z and n

$$r\ s\ z\ \mathbf{zero} = z \quad \text{and}$$
$$r\ s\ z\ (\mathbf{succ}\ n) = s\ n\ (r\ s\ z\ n)$$

(b) Give a closed term **rec**, with its **zero** and **succ**, which is a recursor. (4 marks)

(c) Give a term **pred** such that $\mathbf{pred}\lceil n+1 \rceil = \lceil n \rceil$ and $\mathbf{pred}\lceil 0 \rceil = \lceil 0 \rceil$.

What is the minimum number of reductions necessary to turn $\mathbf{pred}\lceil n \rceil$ into normal form? What is the maximum number of reductions? What reduction strategies lead to each, respectively? (7 marks)

(d) Give a term **fac** such that $\mathbf{fac}\lceil n \rceil = \lceil n! \rceil$.

Give a term **fib** such that $\mathbf{fib}\lceil n \rceil = \lceil fib(n) \rceil$ where $fib(n)$ is the n-th Fibonacci number. (7 marks)

We will now use the λ-calculus to encode lists (a list is a recursive datatype with two constructors: the empty list and cons, which contains a value and a list). We call r a *list recursor* if we have closed terms **nil** and **cons** such that for all terms n, c and l

$$r\ n\ c\ \mathbf{nil} = n \quad \text{and}$$
$$r\ n\ c\ (\mathbf{cons}\ x\ p) = c\ x\ p\ (r\ n\ c\ p)$$

(e) Give a closed term **recList**, with its **nil** and **cons**, which is a list recursor. (4 marks)

We call l a *list* if it is a sequence of **nil**, **cons** and normal forms such that (i) **nil** is not applied to any term (ii) every **cons** is applied to a normal form and then another list. E.g., $\mathbf{expList} \equiv \mathbf{cons}\ \lceil 5 \rceil\ (\mathbf{cons}\ \lceil 2 \rceil\ (\mathbf{cons}\ \lceil 3 \rceil\ \mathbf{nil}))$ is a list.

(f) Give a term **length** which computes the number of **cons** terms in a list, e.g., **length expList** $= \lceil 3 \rceil$.

Give a term **suffixes** which computes all proper suffixes of a list, e.g.,
suffixes expList $=$ **cons** (**cons** $\lceil 2 \rceil$ (**cons** $\lceil 3 \rceil$ **nil**)) (**cons** (**cons** $\lceil 3 \rceil$ **nil**) (**cons nil nil**))

(5 marks)

Lastly, we encode a datatype which corresponds to the construction trees from the lectures (in which bound variables can be represented with de Bruijn indices). We call r a Λ-*tree recursor* if we have closed terms **var**, **app** and **lam** such that for all terms v, a and l.

$$\begin{aligned} r\ v\ a\ l\ (\mathbf{var}\ x) &= v\ x \quad \text{and} \\ r\ v\ a\ l\ (\mathbf{app}\ p\ q) &= a\ p\ (r\ v\ a\ l\ p)\ q\ (r\ v\ a\ l\ q) \\ r\ v\ a\ l\ (\mathbf{lam}\ p) &= l\ p\ (r\ v\ a\ l\ p) \end{aligned}$$

(g) Give a closed term **recΛ**, with its **var**, **app** and **lam**, which is a Λ-tree recursor.

(6 marks)

Question 2

In this question we will study a variation of our type system from Chapter 6. We extend the set of types **Typ** by closing it under the following rule:

$$\frac{A \in \mathbf{Typ} \qquad B \in \mathbf{Typ}}{(A + B) \in \mathbf{Typ}}$$

Type contexts are still sets of pairs of variables with a type. We redefine the relation $\vdash$ as follows:

$$\frac{(x : A) \in \Gamma}{\Gamma \vdash x : A} \qquad \frac{\Gamma \vdash s : B \Rightarrow A \qquad \Gamma \vdash t : B}{\Gamma \vdash st : A} \qquad \frac{\Gamma \cup \{x : B\} \vdash s : A}{\Gamma \vdash \lambda x.s : B \Rightarrow A}$$

$$\frac{\Gamma \vdash s : A \qquad \Gamma \vdash s : B}{\Gamma \vdash s : (A + B)} \qquad \frac{\Gamma \vdash s : (A + B)}{\Gamma \vdash s : A} \qquad \frac{\Gamma \vdash s : (A + B)}{\Gamma \vdash s : B}$$

We can think of $\Gamma \vdash s : (A + B)$ as s having both type A or B in type context Γ. You can assume without proof that $\Gamma \vdash s : A$ implies $\mathrm{FV}(s) \subseteq \mathit{Subjects}(\Gamma)$.

(a) (i) Give a derivation of $\vdash \lambda xy.xxy : (A + (A \Rightarrow (B \Rightarrow C))) \Rightarrow (B \Rightarrow C)$.

(ii) Devise a type A such that $\vdash \lambda xy.y(xy) : A$ is derivable.

(iii) Give an informal argument why Ω is not typable in our extended system.

(6 marks)

Given Γ_0 and Γ_1, we write $\Gamma_0 + \Gamma_1$ for the type context constructed as follows: for any $x \in \mathit{Subjects}(\Gamma_0) \cup \mathit{Subjects}(\Gamma_1)$ we have $(x : A + B) \in \Gamma_0 + \Gamma_1$ in case $(x : A) \in \Gamma_0$ and $(x : B) \in \Gamma_1$, otherwise include $(x : A) \in \Gamma_0 + \Gamma_1$ with the unique type A given by either Γ_0 or Γ_1. Note that any type contexts can be joined in this way. You can assume without proof that $\Gamma \vdash s : A$ implies $\Gamma + \Gamma' \vdash s : A$ for any Γ'.

(b) Show that any term which is in β-normal form is typable in our type system. (5 marks)

(c) Show that our system doesn't enjoy subject expansion, i.e., $s \to_\beta t$ and $\Gamma \vdash t : A$ do not necessarily imply $\Gamma \vdash s : A$. (5 marks)

You can in the following assume without proof the following Subject Construction Lemma:

• If $\Gamma \vdash st : A$, then $A \equiv A_1 + \ldots + A_n$ such that for all $1 \leq i \leq n$ we have $\Gamma \vdash s : B_i \Rightarrow A_i$ and $\Gamma \vdash t : B_i$ for some B_i.

• If $\Gamma \vdash \lambda x.s : A$, then $A \equiv (B_1 \Rightarrow C_1) + \ldots + (B_n \Rightarrow C_n)$ such that for all $1 \leq i \leq n$ we have $\Gamma \cup \{x : B_i\} \vdash s : C_i$.

(d) Show that if $\Gamma \cup \{x : B\} \vdash s : A$ with $x \notin \mathrm{FV}(s)$, then $\Gamma \vdash s : A$. (4 marks)

(e) Show that $s \to_\beta t$, where $s \equiv C[(\lambda x.u)v]$ and $t \equiv C[u[v/x]]$ for some $C[X]$, and $\Gamma \vdash t : A$ as well as $\Gamma \vdash v : B$ for some B imply $\Gamma \vdash s : A$.

Characterise a class of terms for which we can drop the requirement that $\Gamma \vdash v : B$ for some B, i.e., a class of terms which is closed under subject expansion. (10 marks)

(f) Show that any strongly normalisable term is typable in our type system.

You can assume without proof that t being strongly normalisable implies that there is a finite number $l(t)$ equal to the sum of the lengths of all reduction sequences of t. (5 marks)

Question 3

In this exercise you are asked to implement a self-interpreter for the λ-calculus and use it for some meta-programming. Concretely, you will have to do the following:

(a) Define an injective function $\ulcorner - \urcorner : \Lambda \to \Lambda$ which encodes the λ-calculus in itself.

(b) Give a term e which is an *evaluator*, i.e., $\lambda\beta \vdash e\ulcorner s \urcorner = s$ for any closed term s.

(c) Give other terms which perform operations on the encoded terms, e.g., a term **isabs** such that $\lambda\beta \vdash \mathbf{isabs}\ulcorner s \urcorner = \mathbf{t}$ in case $s \equiv \lambda x.t$, etc.

How you solve one part of the exercise influences how you have to solve the other parts—e.g., if you just defined $\ulcorner s \urcorner = s$, then $e \equiv \mathbf{i}$ would do the job in part (b), but you couldn't do anything in part (c). Let's hence disallow such a trivial self-interpreter.

There are several ways in which you can approach this question. The lecture notes already introduced an encoding function $\ulcorner - \urcorner$ which went through a Gödel numbering, which you would have to work out if you followed this approach. Alternatively, you can define $\ulcorner - \urcorner$ directly by understanding Λ as a recursively defined data type—note that in the last part of question 1, we have already defined a recursor for such a data type, which you might find useful here. This approach is also discussed by Herman Geuvers in `https://cs.ioc.ee/ewscs/2016/geuvers/geuvers-slides-lecture2.pdf`. You can use one of these two approaches, or a completely different approach, whichever you like best.

You may describe the self-interpreter just with pen-and-paper, or implement it in a programming language. The marking is independent of which approach and mode of presentation you pick. Your solution will be judged on clarity and correctness of your construction and presentation, as well as originality of your operations in part (c). The page limit for this exercise is 3 pages with reasonable font and formatting. If you implement the interpreter, please put your code in an appendix (which does not count towards the page limit) and describe the crucial parts of the code in the main body of your script. (30 marks)

LAST PAGE